Introduction to Functions, Defining Functions, Functions with multiple parameters, Default Parameters, Keyword Arguments, and Arbitrary Argument Lists, Scope Rules, Higher Order Functions: Map, Filter, and Reduce.

Introduction

Functions are fundamental building blocks in Python programming that enable code reusability, organization, and modularity. A function is a named block of code designed to perform a specific task, which can be called multiple times from different parts of a program. Functions help avoid code repetition, improve readability, make testing easier, and facilitate maintenance of large codebases.

This comprehensive guide covers function definition, parameters and arguments, default parameters, keyword arguments, arbitrary argument lists, scope rules, and higher-order functions including map, filter, and reduce.

Topic 1: Introduction to Functions

1.1 What are Functions?

Functions are reusable blocks of organized code that perform specific tasks. They allow programmers to break down complex problems into smaller, manageable pieces. Functions promote the DRY (Don't Repeat Yourself) principle, making code more maintainable and efficient.

Key benefits of functions:

- **Code Reusability** - Write once, use multiple times throughout the program
- **Modularity** - Break complex problems into smaller, focused units
- **Abstraction** - Hide implementation details, expose only necessary interfaces
- **Maintainability** - Changes in one place automatically apply everywhere the function is called
- **Testing** - Easier to test individual functions in isolation
- **Readability** - Well-named functions make code self-documenting

1.2 Function Terminology

Understanding function terminology is essential for effective programming:

Function Definition: The complete function code including the def keyword, function name, parameters, docstring, and function body.

Function Call: The act of executing a function by using its name followed by parentheses.

Parameters: Variables listed in the function definition that receive values when the function is called.

Arguments: Actual values passed to the function when calling it.

Return Value: The value that a function sends back to the caller using the return statement.

Topic 2: Defining Functions

2.1 Basic Function Syntax

Functions in Python are defined using the def keyword, followed by the function name, parentheses containing optional parameters, and a colon. The function body is indented.

Basic syntax:

```
def function_name(parameters):
"""Docstring describing the function"""
# Function body
# Statements to execute
return value # Optional
```

Simple function without parameters:

```
def greet():
print("Hello, Python!")

greet() # Call the function
```

Output: Hello, Python!

Function without return statement:

```
def display_message():
print("This function doesn't return a value")
print("It only prints messages")

display_message()
```

Functions without an explicit return statement return None by default.

2.2 Functions with Return Values

Functions become more powerful when they return values that can be used in expressions, assignments, or as arguments to other functions.

Function with return value:

```
def add(a, b):
"""Returns the sum of two numbers"""
return a + b

result = add(3, 5)
print(result) # 8
```

Using return value in expression

```
total = add(10, 20) + add(5, 15)
print(total) # 50
```

Multiple return statements:

```
def absolute_value(number):
"""Returns the absolute value of a number"""
if number >= 0:
```

```
return number
else:
return -number

print(absolute_value(5)) # 5
print(absolute_value(-5)) # 5
```

Returning multiple values:

```
def get_min_max(numbers):
"""Returns minimum and maximum values from a list"""
return min(numbers), max(numbers)

minimum, maximum = get_min_max([3, 1, 4, 1, 5, 9, 2, 6])
print(f"Min: {minimum}, Max: {maximum}") # Min: 1, Max: 9
```

2.3 Docstrings

Docstrings are string literals that appear as the first statement in a function, used to document what the function does. They are accessible via the `__doc__` attribute.

Function with docstring:

```
def calculate_area(length, width):
"""
Calculate the area of a rectangle.

Parameters:
    length (float): The length of the rectangle
    width (float): The width of the rectangle

Returns:
    float: The area of the rectangle
"""
return length * width
```

Accessing docstring

```
print(calculate_area.doc)
```

Using the function

```
area = calculate_area(5, 3)
print(f"Area: {area}") # Area: 15
```

Topic 3: Functions with Multiple Parameters

3.1 Defining Multiple Parameters

Functions can accept multiple parameters separated by commas, allowing them to work with several pieces of data simultaneously.

Function with multiple parameters:

```
def describe_person(name, age, city):
"""Prints a description of a person"""
print(f"{name} is {age} years old and lives in {city}.")

describe_person("Alice", 25, "Boston")
```

Output: Alice is 25 years old and lives in Boston.

Function returning calculated value:

```
def calculate_bmi(weight, height):
"""
Calculate Body Mass Index (BMI)

Parameters:
    weight (float): Weight in kilograms
    height (float): Height in meters

Returns:
    float: BMI value
"""
bmi = weight / (height ** 2)
return round(bmi, 2)

bmi = calculate_bmi(70, 1.75)
print(f"BMI: {bmi}") # BMI: 22.86
```

3.2 Parameter Order

Parameters must be provided in the correct order when calling the function, unless keyword arguments are used.

Positional arguments:

```
def display_info(name, age, occupation):
print(f"Name: {name}")
print(f"Age: {age}")
print(f"Occupation: {occupation}")
```

Correct order

```
display_info("Bob", 30, "Engineer")
```

Wrong order produces incorrect output

```
display_info("Engineer", "Bob", 30)
```

3.3 Functions with Mixed Parameter Types

Functions can combine parameters of different data types to handle diverse inputs.

Mixed parameter types:

```
def create_profile(name, age, hobbies, is_active=True):
```

```
"""Creates a user profile dictionary"""
profile = {
'name': name,
'age': age,
'hobbies': hobbies,
'active': is_active
}
return profile

profile = create_profile("Charlie", 28, ["reading", "cycling", "photography"])
print(profile)
```

Topic 4: Default Parameters

4.1 Understanding Default Parameters

Default parameters allow functions to be called with fewer arguments than they are defined to accept. If a value is not provided for a parameter with a default, the default value is used.

Function with default parameters:

```
def greet(name="Guest", message="Welcome"):
"""Greets a person with a custom message"""
print(f"{message}, {name}!")

greet() # Welcome, Guest!
greet("Rahul") # Welcome, Rahul!
greet("Rahul", "Good morning") # Good morning, Rahul!
```

Practical example:

```
def calculate_discount(price, discount_percent=10):
"""
Calculate discounted price

Parameters:
    price (float): Original price
    discount_percent (float): Discount percentage (default: 10)

Returns:
    float: Discounted price
"""
discount = price * (discount_percent / 100)
return price - discount

print(calculate_discount(100)) # 90.0 (10% discount)
print(calculate_discount(100, 20)) # 80.0 (20% discount)
```

4.2 Rules for Default Parameters

Default parameters must follow non-default parameters in the function definition.

Correct placement:

```
def create_user(username, email, role="user", active=True):
"""Creates a user with optional role and status"""
return {
'username': username,
'email': email,
'role': role,
'active': active
}

user1 = create_user("alice", "alice@example.com")
user2 = create_user("bob", "bob@example.com", "admin")
user3 = create_user("charlie", "charlie@example.com", "moderator", False)
```

4.3 Mutable Default Arguments Pitfall

Using mutable objects (like lists or dictionaries) as default parameter values can lead to unexpected behavior because the default value is created only once when the function is defined.

Problem with mutable defaults:

```
def add_item_bad(item, items=[]): # Risky!
"""This demonstrates the mutable default argument pitfall"""
items.append(item)
return items

list1 = add_item_bad("apple")
print(list1) # ['apple']

list2 = add_item_bad("banana")
print(list2) # ['apple', 'banana'] - Unexpected!
```

Correct approach:

```
def add_item(item, items=None):
"""Correctly handles mutable default argument"""
if items is None:
items = []
items.append(item)
return items

list1 = add_item("apple")
print(list1) # ['apple']

list2 = add_item("banana")
print(list2) # ['banana'] - Correct!
```

Topic 5: Keyword Arguments

5.1 Understanding Keyword Arguments

Keyword arguments allow you to specify arguments by parameter name rather than position, improving code readability and allowing arguments to be passed in any order.

Basic keyword arguments:

```
def power(base, exponent):
"""Calculate base raised to exponent"""
return base ** exponent
```

Positional arguments

```
print(power(2, 3)) # 8
```

Keyword arguments

```
print(power(base=2, exponent=3)) # 8
print(power(exponent=3, base=2)) # 8 (order doesn't matter)
```

5.2 Mixing Positional and Keyword Arguments

You can combine positional and keyword arguments, but positional arguments must come before keyword arguments.

Mixed arguments:

```
def create_account(username, email, age, country="USA", language="English"):
"""Creates a user account with mixed arguments"""
return {
'username': username,
'email': email,
'age': age,
'country': country,
'language': language
}
```

Valid combinations

```
account1 = create_account("alice", "alice@example.com", 25)
account2 = create_account("bob", "bob@example.com", 30, country="UK")
account3 = create_account("charlie", "charlie@example.com", 28,
language="Spanish", country="Spain")
```

5.3 Benefits of Keyword Arguments

Keyword arguments provide several advantages:

- **Readability** - Code is self-documenting when parameter names are visible
- **Flexibility** - Arguments can be provided in any order
- **Optional Parameters** - Easy to skip optional parameters in the middle
- **Maintenance** - Adding new parameters doesn't break existing function calls

Example demonstrating flexibility:

```
def format_text(text, bold=False, italic=False, underline=False, color="black"):
"""Formats text with various options"""
formatting = []
if bold:
formatting.append("bold")
if italic:
formatting.append("italic")
if underline:
formatting.append("underline")

result = f"Text: '{text}' - Format: {', '.join(formatting) if formatting else 'none'} - Color: {color}"
return result
```

Skip middle parameters easily

```
print(format_text("Hello", underline=True, color="red"))
```

Topic 6: Arbitrary Argument Lists

6.1 Variable Number of Positional Arguments (*args)

The `*args` parameter allows a function to accept any number of positional arguments, which are collected into a tuple.

**Basic args usage:*

```
def sum_all(*numbers):
"""Returns the sum of all provided numbers"""
total = 0
for num in numbers:
total += num
return total

print(sum_all(1, 2, 3)) # 6
print(sum_all(5, 10, 15, 20)) # 50
print(sum_all(1, 2, 3, 4, 5, 6)) # 21
```

Practical example:

```
def find_maximum(*numbers):
"""Finds the maximum among all provided numbers"""
if not numbers:
return None
return max(numbers)
```

```
print(find_maximum(3, 7, 2, 9, 1)) # 9
print(find_maximum(42)) # 42
print(find_maximum()) # None
```

**Combining regular parameters with args:*

```
def create_profile(name, *hobbies):
"""Creates a profile with name and multiple hobbies"""
print(f"Name: {name}")
print("Hobbies:")
for hobby in hobbies:
print(f" - {hobby}")

create_profile("Alice", "reading", "swimming", "photography")
```

6.2 Variable Number of Keyword Arguments (**kwargs)

The `**kwargs` parameter allows a function to accept any number of keyword arguments, which are collected into a dictionary.

Basic **kwargs usage:

```
def print_profile(**details):
"""Prints profile information from keyword arguments"""
for key, value in details.items():
print(f"{key}: {value}")

print_profile(name="Alice", age=25, city="Boston", occupation="Engineer")
```

Practical example:

```
def create_config(**settings):
"""Creates a configuration dictionary"""
config = {
'version': '1.0',
'default_language': 'English'
}
config.update(settings)
return config

config = create_config(theme="dark", font_size=14, auto_save=True)
print(config)
```

6.3 Combining All Parameter Types

Python allows combining regular parameters, *args, default parameters, and **kwargs in a specific order.

Order of parameters:

```
def demo_function(pos1, pos2, *args, default1="value", default2="another", **kwargs):
"""Demonstrates all parameter types"""
print(f"Positional: {pos1}, {pos2}")
print(f"Extra positional (*args): {args}")
```

```
print(f"Defaults: {default1}, {default2}")
print(f"Keyword arguments (**kwargs): {kwargs}")

demo_function(1, 2, 3, 4, 5, default1="custom", extra1="test", extra2="data")
```

Practical example with all types:

```
def send_email(recipient, subject, *cc_recipients, priority="normal", **options):
"""
Sends an email with various options

Parameters:
    recipient (str): Primary recipient email
    subject (str): Email subject
    *cc_recipients: Additional CC recipients
    priority (str): Email priority level
    **options: Additional email options
"""
print(f"To: {recipient}")
print(f"Subject: {subject}")
print(f"CC: {', '.join(cc_recipients) if cc_recipients else 'None'}")
print(f"Priority: {priority}")
print(f"Options: {options}")

send_email("alice@example.com", "Meeting", "bob@example.com", "charlie@example.com",
priority="high", attachment=True, track_read=True)
```

Topic 7: Scope Rules

7.1 Understanding Scope

Scope refers to the region of code where a variable is visible and accessible. Python uses the LEGB rule to resolve variable names.

LEGB Rule:

- **L - Local**: Variables defined inside the current function
- **E - Enclosing**: Variables in outer functions (for nested functions)
- **G - Global**: Variables defined at the module level
- **B - Built-in**: Python's built-in names (like len, print, etc.)

Basic scope example:

```
x = "global" # Global scope

def outer_function():
x = "enclosing" # Enclosing scope

def inner_function():
    x = "local"  # Local scope
    print(f"Local x: {x}")
```

```
inner_function()
print(f"Enclosing x: {x}")
```

```
outer_function()
print(f"Global x: {x}")
```

Output:

Local x: local

Enclosing x: enclosing

Global x: global

7.2 Local Scope

Variables created inside a function exist only within that function's local scope and are destroyed when the function completes.

Local scope demonstration:

```
def calculate_total():
price = 100 # Local variable
tax = 0.08 # Local variable
total = price + (price * tax)
return total

result = calculate_total()
print(result) # 108.0
```

print(price) # NameError: name 'price' is not defined

Function parameters are local:

```
def greet(name): # 'name' is local to this function
message = f"Hello, {name}!" # 'message' is also local
return message

greeting = greet("Alice")
```

print(name) # NameError

print(message) # NameError

7.3 Global Scope

Variables defined at the module level (outside all functions) are in the global scope and accessible throughout the module.

Global variable access:

```
counter = 0 # Global variable

def increment():
print(f"Counter value: {counter}") # Can read global variable

increment() # Counter value: 0
```

Modifying global variables requires the global keyword:

```
counter = 0

def increment():
global counter # Declare intention to modify global variable
counter += 1
print(f"Counter: {counter}")

increment() # Counter: 1
increment() # Counter: 2
print(counter) # 2
```

Multiple global variables:

```
score = 0
level = 1

def update_game_state(points):
global score, level
score += points
if score >= 100:
level += 1
score = 0

update_game_state(50)
print(f"Score: {score}, Level: {level}") # Score: 50, Level: 1

update_game_state(60)
print(f"Score: {score}, Level: {level}") # Score: 10, Level: 2
```

7.4 Enclosing Scope and Nonlocal

The enclosing scope exists when you have nested functions. The `nonlocal` keyword allows inner functions to modify variables in the enclosing function's scope.

Enclosing scope example:

```
def outer():
count = 0 # Enclosing variable
```

```
def inner():
    nonlocal count  # Modify enclosing variable
    count += 1
    print(f"Count: {count}")

inner()  # Count: 1
inner()  # Count: 2
inner()  # Count: 3
return count
```

```
final_count = outer()
print(f"Final: {final_count}") # Final: 3
```

Practical example with closures:

```
def create_multiplier(factor):
"""Returns a function that multiplies its input by factor"""
def multiply(number):
return number * factor
return multiply

times_3 = create_multiplier(3)
times_5 = create_multiplier(5)

print(times_3(10)) # 30
print(times_5(10)) # 50
```

7.5 Built-in Scope

Python has a set of built-in names that are always available. These can be shadowed by variables in other scopes, though this is generally discouraged.

Built-in names:

len, print, max, min, sum, etc. are in built-in scope

```
numbers = [1, 2, 3, 4, 5]
print(len(numbers)) # Uses built-in len function
```

Shadowing built-ins (avoid this):

```
def bad_example():
len = 10 # Shadows built-in len (bad practice)
print(len) # 10
# print(len([1, 2, 3])) # TypeError: 'int' object is not callable
```

Topic 8: Higher Order Functions

8.1 Understanding Higher Order Functions

A higher-order function is a function that either takes one or more functions as arguments or returns a function as its result. This enables powerful functional programming patterns.

Function as argument:

```
def apply_operation(numbers, operation):
"""Applies an operation to each number in the list"""
result = []
for num in numbers:
result.append(operation(num))
return result

def square(x):
return x * x

def cube(x):
return x * x * x

numbers = [1, 2, 3, 4, 5]
squares = apply_operation(numbers, square)
cubes = apply_operation(numbers, cube)

print(squares) # [1, 4, 9, 16, 25]
print(cubes) # [1, 8, 27, 64, 125]
```

Function returning function:

```
def create_multiplier(factor):
"""Returns a function that multiplies by factor"""
def multiplier(x):
return x * factor
return multiplier

double = create_multiplier(2)
triple = create_multiplier(3)

print(double(5)) # 10
print(triple(5)) # 15
```

8.2 Lambda Functions

Lambda functions are small anonymous functions defined using the `lambda` keyword. They are often used with higher-order functions for concise code.

Lambda syntax:

Regular function

```
def add(x, y):
return x + y
```

Equivalent lambda

```
add_lambda = lambda x, y: x + y

print(add(3, 5)) # 8
print(add_lambda(3, 5)) # 8
```

Lambda with single argument:

```
square = lambda x: x ** 2
print(square(4)) # 16
```

Lambda in higher-order functions:

```
numbers = [1, 2, 3, 4, 5]
squares = list(map(lambda x: x ** 2, numbers))
print(squares) # [1, 4, 9, 16, 25]
```

8.3 The map() Function

The map() function applies a given function to each item of an iterable and returns an iterator of the results.

Basic map usage:

```
def fahrenheit_to_celsius(f):
return (f - 32) * 5/9

temperatures_f = [32, 68, 100, 212]
temperatures_c = list(map(fahrenheit_to_celsius, temperatures_f))
print(temperatures_c) # [0.0, 20.0, 37.77..., 100.0]
```

Map with lambda:

```
numbers = [1, 2, 3, 4, 5]
doubled = list(map(lambda x: x * 2, numbers))
print(doubled) # [2, 4, 6, 8, 10]
```

Map with multiple iterables:

```
numbers1 = [1, 2, 3, 4]
numbers2 = [10, 20, 30, 40]
sums = list(map(lambda x, y: x + y, numbers1, numbers2))
print(sums) # [11, 22, 33, 44]
```

Practical example:

```
names = ["alice", "bob", "charlie"]
capitalized = list(map(str.capitalize, names))
print(capitalized) # ['Alice', 'Bob', 'Charlie']
```

8.4 The filter() Function

The filter() function constructs an iterator from elements of an iterable for which a function returns True.

Basic filter usage:

```
def is_even(n):
return n % 2 == 0

numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
even_numbers = list(filter(is_even, numbers))
print(even_numbers) # [2, 4, 6, 8, 10]
```

Filter with lambda:

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
greater_than_five = list(filter(lambda x: x > 5, numbers))
print(greater_than_five) # [6, 7, 8, 9, 10]
```

Filtering strings:

```
words = ["apple", "banana", "cherry", "date", "elderberry"]
long_words = list(filter(lambda w: len(w) > 5, words))
print(long_words) # ['banana', 'cherry', 'elderberry']
```

Practical example:

```
students = [
{"name": "Alice", "score": 85},
{"name": "Bob", "score": 72},
{"name": "Charlie", "score": 90},
{"name": "David", "score": 65}
]

passing_students = list(filter(lambda s: s["score"] >= 75, students))
for student in passing_students:
print(f"{student['name']}: {student['score']}")
```

8.5 The reduce() Function

The reduce() function applies a binary function cumulatively to items of an iterable, reducing it to a single value. It is available in the functools module.

Basic reduce usage:

```
from functools import reduce

def add(x, y):
return x + y

numbers = [1, 2, 3, 4, 5]
total = reduce(add, numbers)
print(total) # 15
```

Reduce with lambda:

```
from functools import reduce
```

```
numbers = [1, 2, 3, 4, 5]
product = reduce(lambda x, y: x * y, numbers)
print(product) # 120
```

Reduce with initial value:

```
from functools import reduce

numbers = [1, 2, 3, 4, 5]
total = reduce(lambda x, y: x + y, numbers, 10) # Start with 10
print(total) # 25 (10 + 1 + 2 + 3 + 4 + 5)
```

Finding maximum:

```
from functools import reduce

numbers = [3, 7, 2, 9, 1, 5]
maximum = reduce(lambda x, y: x if x > y else y, numbers)
print(maximum) # 9
```

8.6 Combining map, filter, and reduce

These functions can be combined to create powerful data processing pipelines.

Example 1: Sum of squares of even numbers

```
from functools import reduce

numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

Filter even numbers

```
evens = filter(lambda x: x % 2 == 0, numbers)
```

Square them

```
squares = map(lambda x: x ** 2, evens)
```

Sum them

```
result = reduce(lambda x, y: x + y, squares)

print(result) # 220 (4 + 16 + 36 + 64 + 100)
```

Example 2: Processing text data

```
from functools import reduce

sentences = [
"Hello world",
"Python programming",
"Higher order functions"
]
```

Split into words

```
all_words = reduce(lambda acc, sentence: acc + sentence.split(), sentences, [])
```

Filter words longer than 5 characters

```
long_words = list(filter(lambda w: len(w) > 5, all_words))
```

Convert to uppercase

```
upper_words = list(map(str.upper, long_words))

print(upper_words) # ['PYTHON', 'PROGRAMMING', 'HIGHER', 'FUNCTIONS']
```

Example 3: Chained operations

```
from functools import reduce

# Calculate average of squares of positive numbers
numbers = [-2, 3, -1, 4, 5, -3, 6]

positive = list(filter(lambda x: x > 0, numbers))
squares = list(map(lambda x: x ** 2, positive))
total = reduce(lambda x, y: x + y, squares)
average = total / len(squares)

print(f"Average of squares: {average}") # 20.5
```

Best Practices

1. **Use descriptive function names** - Function names should clearly indicate what the function does (use verbs)
2. **Keep functions focused** - Each function should do one thing and do it well (Single Responsibility Principle)
3. **Use docstrings** - Document all functions with clear descriptions, parameter types, and return values
4. **Avoid side effects** - Functions should preferably return values rather than modify global state
5. **Use default parameters wisely** - Avoid mutable defaults; use None and initialize inside the function
6. **Prefer keyword arguments** - For functions with many parameters, use keyword arguments for clarity
7. **Limit function length** - Keep functions short (typically under 20-30 lines)
8. **Use type hints** - In Python 3.5+, use type hints to document expected types
9. **Handle errors appropriately** - Validate inputs and handle potential errors gracefully
10. **Avoid deep nesting** - Extract nested logic into separate functions for better readability

Common Pitfalls to Avoid

- **Mutable default arguments** - Never use lists or dictionaries as default parameter values
- **Modifying global state** - Avoid excessive use of global variables; pass data through parameters

- **Too many parameters** - If a function needs many parameters, consider using a dictionary or object
- **Unclear function names** - Avoid vague names like do_stuff or process_data
- **Functions that do too much** - Break complex functions into smaller, focused functions
- **Not returning values** - Prefer returning values over printing or modifying global state
- **Shadowing built-ins** - Avoid using names like list, dict, str, len as variable or function names
- **Ignoring return values** - Always use or store the return value of functions that return something
- **Complex lambda functions** - Keep lambdas simple; use regular functions for complex logic
- **Overusing map/filter/reduce** - List comprehensions are often more Pythonic and readable

Summary

This comprehensive guide covered Python functions from basic definitions to advanced concepts including default parameters, keyword arguments, arbitrary argument lists, scope rules, and higher-order functions. Functions are fundamental to writing clean, maintainable, and reusable code in Python.

Key takeaways include understanding the LEGB scope resolution rule, properly using default and keyword arguments, leveraging *args and **kwargs for flexible function interfaces, and mastering higher-order functions like map, filter, and reduce for functional programming patterns. Proper use of functions leads to more modular, testable, and maintainable code that follows best practices and avoids common pitfalls.

Practice Problems

Section A: Basic Functions and Parameters

Problem 1: Write a function called `calculate_rectangle_area` that takes length and width as parameters and returns the area. Include a docstring.

Problem 2: Create a function `is_prime` that checks if a given number is prime. Return `True` if prime, `False` otherwise.

Problem 3: Write a function `factorial` that calculates the factorial of a number using recursion.

Problem 4: Create a function `reverse_string` that takes a string and returns it reversed without using slicing.

Problem 5: Write a function `find_largest` that takes three numbers as parameters and returns the largest one without using the `max()` function.

Problem 6: Create a function `count_vowels` that counts the number of vowels in a given string.

Section B: Default Parameters and Keyword Arguments

Problem 7: Write a function `greet_user` with parameters `name`, `greeting="Hello"`, and `punctuation="!"`. Test it with different combinations of arguments.

Problem 8: Create a function `calculate_bill` that takes `amount`, `tax_rate=0.08`, and `tip_percent=15` as parameters and returns the total bill.

Problem 9: Write a function create_email with parameters username, domain="example.com", and extension="com" that constructs an email address.

Problem 10: Create a function format_name that takes first_name, last_name, and optional parameters middle_name=None and title=None to format a full name.

Section C: Arbitrary Arguments (*args and **kwargs)

Problem 11: Write a function calculate_average that accepts any number of arguments and returns their average.

Problem 12: Create a function find_min_max using *args that returns a tuple containing the minimum and maximum values.

Problem 13: Write a function concatenate_strings that takes a separator and any number of strings using *args, and returns them joined by the separator.

Problem 14: Create a function build_profile that takes first_name, last_name, and **user_info to build a user profile dictionary.

Problem 15: Write a function flexible_multiply that takes factor and *numbers, returning a list of all numbers multiplied by the factor.

Problem 16: Create a function create_query_string that takes a base URL and **params to construct a URL query string (e.g., "url?key1=value1&key2=value2").

Section D: Scope and Closures

Problem 17: Write code demonstrating the LEGB rule by creating variables with the same name in local, enclosing, global, and built-in scopes.

Problem 18: Create a function counter that uses a closure to maintain a count. Each call to the returned function should increment and return the count.

Problem 19: Write a function create_power_function that takes an exponent and returns a function that raises its argument to that exponent.

Problem 20: Create a function that demonstrates the difference between using global and not using it when trying to modify a global variable.

Problem 21: Write a nested function structure where an inner function modifies an enclosing variable using nonlocal.

Section E: Lambda Functions

Problem 22: Create lambda functions for the following operations: add 10 to a number, check if a number is even, convert Celsius to Fahrenheit.

Problem 23: Write a lambda function that takes two arguments and returns the larger one.

Problem 24: Use a lambda function with sorted() to sort a list of tuples by the second element.

Problem 25: Create a list of lambda functions (each multiplying by 1, 2, 3, 4, 5) and use them to process a number.

Section F: Higher Order Functions - map()

Problem 26: Use map() to convert a list of strings to integers.

Problem 27: Apply map() with a lambda function to square all numbers in a list.

Problem 28: Use map() with multiple lists to add corresponding elements from two lists.

Problem 29: Create a list of dictionaries with names and use map() to extract just the names.

Problem 30: Use map() to convert a list of temperatures from Celsius to Fahrenheit.

Section G: Higher Order Functions - filter()

Problem 31: Use filter() to extract all even numbers from a list.

Problem 32: Filter a list of strings to keep only those longer than 5 characters.

Problem 33: Use filter() to find all numbers divisible by both 3 and 5 from a range.

Problem 34: Filter a list of dictionaries to keep only students with scores above 80.

Problem 35: Use filter() with a custom function to find all palindrome strings in a list.

Section H: Higher Order Functions - reduce()

Problem 36: Use reduce() to find the product of all numbers in a list.

Problem 37: Apply reduce() to find the maximum number in a list without using max().

Problem 38: Use reduce() to concatenate all strings in a list into a single string.

Problem 39: Apply reduce() to flatten a list of lists into a single list.

Problem 40: Use reduce() with an initial value to calculate the sum of numbers starting from 100.

Section I: Combining map, filter, and reduce

Problem 41: Find the sum of cubes of all odd numbers from 1 to 20 using filter(), map(), and reduce().

Problem 42: Given a list of sentences, find the total length of all words longer than 4 characters.

Problem 43: Filter positive numbers, square them, and find their average using higher-order functions.

Problem 44: From a list of strings, filter palindromes, convert to uppercase, and join them with commas.

Problem 45: Given a list of numbers, remove duplicates, keep only even numbers, and calculate their product.

Section J: Advanced Function Concepts

Problem 46: Write a decorator function that prints execution time of any function it decorates.

Problem 47: Create a function that returns different functions based on a string parameter ("add", "subtract", "multiply", "divide").

Problem 48: Implement a memoization decorator that caches function results for expensive computations.

Problem 49: Write a function compose that takes two functions and returns their composition (f(g(x))).

Problem 50: Create a generator function that yields Fibonacci numbers indefinitely.

Solutions to Practice Problems

Section A Solutions: Basic Functions and Parameters

Problem 1: Calculate rectangle area

```
def calculate_rectangle_area(length, width):
"""
Calculate the area of a rectangle.

Parameters:
    length (float): The length of the rectangle
    width (float): The width of the rectangle

Returns:
    float: The area of the rectangle
"""
return length * width

print(calculate_rectangle_area(5, 3)) # 15
```

Problem 2: Check prime number

```
def is_prime(n):
"""Check if a number is prime"""
if n < 2:
return False
for i in range(2, int(n ** 0.5) + 1):
if n % i == 0:
return False
return True

print(is_prime(17)) # True
print(is_prime(20)) # False
```

Problem 3: Factorial using recursion

```
def factorial(n):
"""Calculate factorial using recursion"""
if n == 0 or n == 1:
return 1
return n * factorial(n - 1)
```

```
print(factorial(5)) # 120
print(factorial(0)) # 1
```

Problem 4: Reverse string without slicing

```
def reverse_string(s):
"""Reverse a string without using slicing"""
reversed_str = ""
for char in s:
reversed_str = char + reversed_str
return reversed_str

print(reverse_string("hello")) # "olleh"
```

Problem 5: Find largest of three numbers

```
def find_largest(a, b, c):
"""Find the largest of three numbers"""
if a >= b and a >= c:
return a
elif b >= a and b >= c:
return b
else:
return c

print(find_largest(10, 25, 15)) # 25
```

Problem 6: Count vowels in string

```
def count_vowels(s):
"""Count vowels in a string"""
vowels = "aeiouAEIOU"
count = 0
for char in s:
if char in vowels:
count += 1
return count

print(count_vowels("Hello World")) # 3
```

Section B Solutions: Default Parameters and Keyword Arguments

Problem 7: Greet user with defaults

```
def greet_user(name, greeting="Hello", punctuation="!"):
"""Greet a user with customizable greeting"""
return f"{greeting}, {name}{punctuation}"

print(greet_user("Alice")) # Hello, Alice!
print(greet_user("Bob", "Hi")) # Hi, Bob!
print(greet_user("Charlie", "Good morning", ".")) # Good morning, Charlie.
print(greet_user("David", punctuation="!!!")) # Hello, David!!!
```

Problem 8: Calculate bill with defaults

```
def calculate_bill(amount, tax_rate=0.08, tip_percent=15):
"""Calculate total bill with tax and tip"""
tax = amount * tax_rate
tip = amount * (tip_percent / 100)
total = amount + tax + tip
return round(total, 2)

print(calculate_bill(100)) # 123.0
print(calculate_bill(100, 0.10)) # 125.0
print(calculate_bill(100, 0.08, 20)) # 128.0
```

Problem 9: Create email address

```
def create_email(username, domain="example.com", extension="com"):
"""Construct an email address"""
return f"{username}@{domain}"

print(create_email("alice")) # alice@example.com
print(create_email("bob", "gmail.com")) # bob@gmail.com
print(create_email("charlie", "company.org")) # charlie@company.org
```

Problem 10: Format name with optional parts

```
def format_name(first_name, last_name, middle_name=None, title=None):
"""Format a full name with optional middle name and title"""
parts = []
if title:
parts.append(title)
parts.append(first_name)
if middle_name:
parts.append(middle_name)
parts.append(last_name)
return " ".join(parts)

print(format_name("John", "Doe")) # John Doe
print(format_name("Jane", "Smith", middle_name="Marie")) # Jane Marie Smith
print(format_name("Bob", "Johnson", title="Dr.")) # Dr. Bob Johnson
print(format_name("Alice", "Brown", "Kay", "Ms.")) # Ms. Alice Kay Brown
```

Section C Solutions: Arbitrary Arguments

Problem 11: Calculate average of any numbers

```
def calculate_average(*numbers):
"""Calculate average of any number of arguments"""
if not numbers:
return 0
return sum(numbers) / len(numbers)

print(calculate_average(10, 20, 30)) # 20.0
print(calculate_average(5, 15, 25, 35)) # 20.0
```

Problem 12: Find min and max using args

```
def find_min_max(*numbers):
"""Return tuple of minimum and maximum values"""
if not numbers:
return None
return (min(numbers), max(numbers))

print(find_min_max(3, 7, 2, 9, 1, 5)) # (1, 9)
```

Problem 13: Concatenate strings with separator

```
def concatenate_strings(separator, *strings):
"""Join strings with given separator"""
return separator.join(strings)

print(concatenate_strings(", ", "apple", "banana", "orange"))
```

apple, banana, orange

```
print(concatenate_strings(" - ", "one", "two", "three"))
```

one - two - three

Problem 14: Build profile with **kwargs

```
def build_profile(first_name, last_name, **user_info):
"""Build a user profile dictionary"""
profile = {
'first_name': first_name,
'last_name': last_name
}
profile.update(user_info)
return profile

profile = build_profile("Albert", "Einstein",
location="Princeton",
field="Physics",
age=76)
print(profile)
```

Problem 15: Flexible multiply

```
def flexible_multiply(factor, *numbers):
"""Multiply all numbers by factor"""
return [factor * num for num in numbers]

print(flexible_multiply(3, 1, 2, 3, 4, 5))
```

[3, 6, 9, 12, 15]

Problem 16: Create query string

```
def create_query_string(base_url, **params):
"""Construct URL query string from parameters"""
query_parts = [f"{key}={value}" for key, value in params.items()]
```

```
query_string = "&".join(query_parts)
return f"{base_url}?{query_string}"

url = create_query_string("https://api.example.com/search",
query="python",
page=1,
limit=10)
print(url)
```

https://api.example.com/search?query=python&page=1&limit=10

Section D Solutions: Scope and Closures

Problem 17: Demonstrate LEGB rule

```
x = "global x" # Global

def outer():
x = "enclosing x" # Enclosing

def inner():
    x = "local x"  # Local
    print("Local:", x)
    print("Built-in len:", len([1, 2, 3]))  # Built-in

inner()
print("Enclosing:", x)

outer()
print("Global:", x)
```

Problem 18: Counter closure

```
def counter():
"""Create a counter using closure"""
count = 0

def increment():
    nonlocal count
    count += 1
    return count

return increment

counter1 = counter()
print(counter1()) # 1
print(counter1()) # 2
print(counter1()) # 3

counter2 = counter()
print(counter2()) # 1
```

Problem 19: Create power function

```
def create_power_function(exponent):
"""Return a function that raises to given exponent"""
def power(base):
return base ** exponent
return power

square = create_power_function(2)
cube = create_power_function(3)

print(square(5)) # 25
print(cube(3)) # 27
```

Problem 20: Global keyword demonstration

```
count = 0

def increment_wrong():
"""Tries to modify global without global keyword"""
# count += 1 # This would raise UnboundLocalError
pass

def increment_correct():
"""Correctly modifies global variable"""
global count
count += 1

increment_correct()
print(count) # 1
increment_correct()
print(count) # 2
```

Problem 21: Nonlocal demonstration

```
def outer_function():
"""Demonstrate nonlocal keyword"""
x = 10

def inner_function():
    nonlocal x
    x += 5
    print(f"Inner x: {x}")

print(f"Before: {x}")
inner_function()
print(f"After: {x}")

outer_function()
```

Before: 10

Inner x: 15

After: 15

Section E Solutions: Lambda Functions

Problem 22: Basic lambda functions

```
add_10 = lambda x: x + 10
is_even = lambda x: x % 2 == 0
celsius_to_fahrenheit = lambda c: (c * 9/5) + 32

print(add_10(5)) # 15
print(is_even(4)) # True
print(celsius_to_fahrenheit(25)) # 77.0
```

Problem 23: Lambda returning larger value

```
max_of_two = lambda x, y: x if x > y else y

print(max_of_two(10, 20)) # 20
print(max_of_two(15, 8)) # 15
```

Problem 24: Sort tuples by second element

```
data = [(1, 5), (3, 2), (2, 8), (4, 1)]
sorted_data = sorted(data, key=lambda x: x[1])
print(sorted_data)
```

[(4, 1), (3, 2), (1, 5), (2, 8)]

Problem 25: List of lambda functions

```
multipliers = [lambda x, n=i: x * n for i in range(1, 6)]

number = 10
for func in multipliers:
print(func(number), end=" ")
```

10 20 30 40 50

Section F Solutions: map()

Problem 26: Convert strings to integers

```
str_numbers = ["1", "2", "3", "4", "5"]
int_numbers = list(map(int, str_numbers))
print(int_numbers) # [1, 2, 3, 4, 5]
```

Problem 27: Square all numbers

```
numbers = [1, 2, 3, 4, 5]
squares = list(map(lambda x: x ** 2, numbers))
print(squares) # [1, 4, 9, 16, 25]
```

Problem 28: Add corresponding elements

```
list1 = [1, 2, 3, 4]
```

```
list2 = [10, 20, 30, 40]
sums = list(map(lambda x, y: x + y, list1, list2))
print(sums) # [11, 22, 33, 44]
```

Problem 29: Extract names from dictionaries

```
people = [
{"name": "Alice", "age": 25},
{"name": "Bob", "age": 30},
{"name": "Charlie", "age": 35}
]
names = list(map(lambda p: p["name"], people))
print(names) # ['Alice', 'Bob', 'Charlie']
```

Problem 30: Convert Celsius to Fahrenheit

```
celsius_temps = [0, 10, 20, 30, 100]
fahrenheit_temps = list(map(lambda c: (c * 9/5) + 32, celsius_temps))
print(fahrenheit_temps) # [32.0, 50.0, 68.0, 86.0, 212.0]
```

Section G Solutions: filter()

Problem 31: Extract even numbers

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
evens = list(filter(lambda x: x % 2 == 0, numbers))
print(evens) # [2, 4, 6, 8, 10]
```

Problem 32: Filter long strings

```
words = ["hi", "hello", "world", "python", "code"]
long_words = list(filter(lambda w: len(w) > 5, words))
print(long_words) # ['python']
```

Problem 33: Numbers divisible by 3 and 5

```
numbers = range(1, 101)
divisible = list(filter(lambda x: x % 3 == 0 and x % 5 == 0, numbers))
print(divisible) # [15, 30, 45, 60, 75, 90]
```

Problem 34: Filter high-scoring students

```
students = [
{"name": "Alice", "score": 85},
{"name": "Bob", "score": 72},
{"name": "Charlie", "score": 90},
{"name": "David", "score": 78}
]
high_scorers = list(filter(lambda s: s["score"] > 80, students))
for student in high_scorers:
print(f"{student['name']}: {student['score']}")
```

Alice: 85

Charlie: 90

Problem 35: Filter palindromes

```
def is_palindrome(s):
return s == s[::-1]

words = ["radar", "hello", "level", "world", "civic", "python"]
palindromes = list(filter(is_palindrome, words))
print(palindromes) # ['radar', 'level', 'civic']
```

Section H Solutions: reduce()

Problem 36: Product of all numbers

```
from functools import reduce

numbers = [1, 2, 3, 4, 5]
product = reduce(lambda x, y: x * y, numbers)
print(product) # 120
```

Problem 37: Find maximum using reduce

```
from functools import reduce

numbers = [3, 7, 2, 9, 1, 5, 8]
maximum = reduce(lambda x, y: x if x > y else y, numbers)
print(maximum) # 9
```

Problem 38: Concatenate strings

```
from functools import reduce

words = ["Hello", " ", "World", " ", "from", " ", "Python"]
sentence = reduce(lambda x, y: x + y, words)
print(sentence) # Hello World from Python
```

Problem 39: Flatten list of lists

```
from functools import reduce

nested_list = [[1, 2], [3, 4], [5, 6], [7, 8]]
flat_list = reduce(lambda x, y: x + y, nested_list)
print(flat_list) # [1, 2, 3, 4, 5, 6, 7, 8]
```

Problem 40: Sum with initial value

```
from functools import reduce

numbers = [1, 2, 3, 4, 5]
total = reduce(lambda x, y: x + y, numbers, 100)
print(total) # 115
```

Section I Solutions: Combining Functions

Problem 41: Sum of cubes of odd numbers

```
from functools import reduce
```

```
numbers = range(1, 21)
odds = filter(lambda x: x % 2 != 0, numbers)
cubes = map(lambda x: x ** 3, odds)
result = reduce(lambda x, y: x + y, cubes)
print(result) # 44100
```

Problem 42: Total length of long words

```
from functools import reduce

sentences = ["Hello world", "Python programming", "Map filter reduce"]
all_words = reduce(lambda acc, s: acc + s.split(), sentences, [])
long_words = filter(lambda w: len(w) > 4, all_words)
lengths = map(len, long_words)
total_length = reduce(lambda x, y: x + y, lengths, 0)
print(total_length) # 35
```

Problem 43: Average of squares of positive numbers

```
from functools import reduce

numbers = [-2, 3, -1, 4, 5, -3, 6]
positive = list(filter(lambda x: x > 0, numbers))
squares = list(map(lambda x: x ** 2, positive))
total = reduce(lambda x, y: x + y, squares)
average = total / len(squares)
print(f"Average: {average}") # 20.5
```

Problem 44: Process palindromes

```
from functools import reduce

def is_palindrome(s):
return s.lower() == s.lower()[::-1]

words = ["Radar", "hello", "Level", "world", "Civic", "python"]
palindromes = filter(is_palindrome, words)
uppercase = map(str.upper, palindromes)
result = reduce(lambda x, y: f"{x}, {y}", uppercase)
print(result) # RADAR, LEVEL, CIVIC
```

Problem 45: Product of unique even numbers

```
from functools import reduce

numbers = [1, 2, 3, 2, 4, 5, 4, 6, 7, 6, 8]
unique = list(set(numbers))
evens = filter(lambda x: x % 2 == 0, unique)
product = reduce(lambda x, y: x * y, evens, 1)
print(product) # 384
```

Section J Solutions: Advanced Concepts

Problem 46: Execution time decorator

```
import time

def timer(func):
"""Decorator to measure execution time"""
def wrapper(*args, **kwargs):
start = time.time()
result = func(*args, **kwargs)
end = time.time()
print(f"{func.name} took {end - start:.4f} seconds")
return result
return wrapper

@timer
def slow_function():
time.sleep(1)
return "Done"

slow_function()
```

Problem 47: Function factory

```
def get_operation(operation):
"""Return appropriate function based on operation string"""
operations = {
"add": lambda x, y: x + y,
"subtract": lambda x, y: x - y,
"multiply": lambda x, y: x * y,
"divide": lambda x, y: x / y if y != 0 else "Error: Division by zero"
}
return operations.get(operation, lambda x, y: "Invalid operation")

add_func = get_operation("add")
print(add_func(5, 3)) # 8

multiply_func = get_operation("multiply")
print(multiply_func(4, 7)) # 28
```

Problem 48: Memoization decorator

```
def memoize(func):
"""Cache function results for efficiency"""
cache = {}

def wrapper(*args):
    if args in cache:
        print(f"Returning cached result for {args}")
        return cache[args]
    result = func(*args)
    cache[args] = result
    return result
```

```
return wrapper
```

```
@memoize
def fibonacci(n):
"""Calculate nth Fibonacci number"""
if n < 2:
return n
return fibonacci(n - 1) + fibonacci(n - 2)

print(fibonacci(10)) # 55
print(fibonacci(10)) # Returns cached result
```

Problem 49: Function composition

```
def compose(f, g):
"""Return composition of two functions: f(g(x))"""
def composed(x):
return f(g(x))
return composed

def add_5(x):
return x + 5

def multiply_3(x):
return x * 3
```

Compose: first multiply by 3, then add 5

```
composed_func = compose(add_5, multiply_3)
print(composed_func(10)) # 35 (10 * 3 + 5)
```

Problem 50: Fibonacci generator

```
def fibonacci_generator():
"""Generate Fibonacci numbers indefinitely"""
a, b = 0, 1
while True:
yield a
a, b = b, a + b
```

Generate first 10 Fibonacci numbers

```
fib = fibonacci_generator()
for _ in range(10):
print(next(fib), end=" ")
```

0 1 1 2 3 5 8 13 21 34